

Sistemas Distribuídos

Exame¹

23 de janeiro de 2026

Duração: 2h00m

Responda a cada grupo numa folha separada, entregando obrigatoriamente três folhas.

I

Responda diretamente a cada pergunta e com grafia bem legível. Omita preâmbulos e considerações genéricas sobre cada um dos assuntos.

- 1 Neste UC não usamos `synchronized` que permite fazer exclusão mútua mas obrigando a que o primeiro trinco libertado seja sempre o último adquirido (LIFO). Porque é que isto é importante?
- 2 Como funciona e em que medida pode a migração de código ser útil em sistemas de gestão de bases de dados?
- 3 Explique porque é que num algoritmo de replicação com *quorum* é necessário que $n_r + n_w > n$ (onde n_r é o quorum de leitura, n_w é o quorum de escritas e n é o número de réplicas).
- 4 Que modelo de *threading* recomendaria para um servidor Chord? Justifique.

II

Considere um sistema cliente/servidor de apoio à gestão da ocupação de S salas de reunião (sala 0 até sala $S - 1$). Cada reunião é designada por um identificador único previamente escolhido. Uma sala livre poderá ser atribuída a uma reunião, ficando novamente livre quando a reunião termina (todos os participantes tiverem saído da sala). Depois de uma reunião começar só podem entrar participantes mais tarde se não estiverem pessoas à espera para outras reuniões. Estando pessoas à espera para diferentes reuniões, quando uma sala fica livre deve ser dada prioridade por ordem de chegada da primeira pessoa que apareceu para a reunião. Apresente uma classe Java (para ser usada no servidor) que implemente a interface abaixo, tendo em conta que os seus métodos serão invocados num ambiente *multi-threaded*. Minimize as *threads* que são acordadas.

```
interface Manager {  
    int join(String identifier) throws InterruptedException;  
    void leave(String identifier);  
    List<String> meetings();  
}
```

O método `join` deve bloquear até o participante poder entrar numa sala atribuída à reunião, retornando o número da sala, ou -1 caso já não possa entrar. O método `leave` serve para informar sobre o abandono da sala pelo participante da reunião. O método `meetings` deve devolver a lista das reuniões a decorrer.

III

Considere um serviço ao qual clientes se ligam por TCP, para participarem numa reunião. Ao chegar, um cliente envia o identificador da reunião, devendo o servidor responder com `ENTER n`, sendo n o número da sala, quando a sala estiver disponível, ou `LATE` se já não puder entrar na reunião. Se entrou numa reunião, o cliente poderá enviar `LEAVE`, indicando que abandonou a sala. Em qualquer momento, mesmo à espera, um cliente deverá poder enviar `MEETINGS`, o qual deverá ser respondido com a lista das reuniões em curso. Implemente só o programa servidor usando *threads*, *sockets* TCP, e assumindo a existência da classe desenvolvida na pergunta anterior. Use um protocolo o mais simples possível, por exemplo, baseado em linhas de texto.

¹Cotação — $4 \times 2.5 + 7 + 3$